Challenge 1: Autonomous Content-to-Action Agent (Insight → Action System)

Challenge Overview

In today's world, organizations are flooded with unstructured information — reports, news, dashboards, policy updates — but most AI systems stop at **summarization or analysis**.

Real-world systems must go further.

They must:

- understand information
- extract insights
- make decisions
- and **take action**

Problem Statement

Build an **Agentic AI System** that transforms **unstructured content into actionable outcomes**.

Your system must:

1. Ingest unstructured input (text, reports, articles, dashboards)
2. Extract key insights
3. Analyze implications
4. Generate recommended actions
5. **Simulate execution of at least one action**
6. Show the resulting system state or outcome

Mandatory Requirement: Google Antigravity

Teams **MUST use Google Antigravity** as the core development platform to:

- orchestrate agent workflows
- manage reasoning and planning
- integrate tools/APIs
- handle execution of actions

Use of additional LLMs is allowed, but **Antigravity must be central to system logic**.

Example Use Cases

Teams may choose any domain such as:

- Business operations
- Public policy
- Logistics / supply chain
- Financial insights
- News analysis
- Urban systems

Example Scenario 1: Business Insight

Input:

A sales report showing declining orders in a region

Output:

Insight:
Orders declined by 25% in Lahore
Impact:
Revenue loss expected
Recommended Action:
Launch regional discount campaign
Simulated Execution:
- Campaign created
- Notification generated
Result:
Projected reach: 5,000 users

Example Scenario 2: Policy / News

Input:

News article about fuel price increase

Output:

Insight:
Fuel prices rising
Impact:
Delivery costs increasing
Recommended Action:
Update delivery pricing
Simulated Execution:
- Pricing table updated
- Customer notification drafted

Result:
New delivery cost applied in checkout simulation

System Requirements

Your solution must include:

1. Content Understanding

- Process unstructured input (text, PDF, website, new article etc.)
- Extract key facts and signals

2. Insight Extraction

- Identify meaningful patterns or insights
- Avoid generic summarization

3. Impact Analysis

- Explain **why the insight matters**
- Connect insight to real-world consequences

4. Action Generation

- Generate **clear, actionable recommendations**
- Must be realistic and domain-relevant

5. Action Simulation (CRITICAL REQUIREMENT)

You must simulate execution of at least **one action**.

Simulation can include:

- Mock API call
- Dashboard update
- CRM / spreadsheet update
- Notification system
- Email/SMS generation
- Workflow trigger

6. Outcome Visualization

Show:

- before vs after state
- logs of action execution
- resulting system change

7. Agentic Workflow

Your system must demonstrate:

- multiple agents OR structured reasoning steps
- planning and execution flow
- traceable decision-making

Deliverables

Teams must submit:

1. Working Prototype with Mobile App (MUST) and Web App (Optional)

2. Demo Video (3–5 minutes)

Must clearly show:

- input → insight → action → simulation → result

3. Agent Trace / Logs from Antigravity

- Workplan
- Tasks Plan
- reasoning steps
- decision flow
- action execution

4. Documentation (README)

Include:

- architecture overview
- tools/APIs used
- how Antigravity is used
- assumptions

Evaluation Criteria

1. Use of Google Antigravity — 25%

- Antigravity used in core orchestration
- Effective handling of reasoning and tool execution
- Not used superficially

2. Agentic Reasoning & Workflow — 20%

- Clear multi-step reasoning
- Logical flow: insight → action → execution
- Evidence of autonomy

3. Insight & Decision Quality — 20%

- Insights are meaningful and non-trivial
- Strong reasoning behind actions
- Outputs are structured and clear

4. Action Simulation & Outcome — 15%

- Action is realistically simulated
- Clear demonstration of system state change
- End-to-end execution visible

5. Technical Implementation — 10%

- Clean architecture
- Tool/API integration
- Robustness and handling of edge cases

6. Innovation & UX — 10%

- Creative approach
- Intuitive interface
- Clear and compelling demo

Important Guidelines

- This is NOT a summarization tool
- Systems must demonstrate **action-oriented intelligence**
- At least one action must be **executed in simulation**
- Use of **real-world inspired scenarios is encouraged**
- No sensitive or real personal data should be used

Challenge 2: AI Service Orchestrator for Informal Economy

Challenge Overview

Informal economy, including plumbers, electricians, tutors, beauticians, and home service providers, operates largely through:

- WhatsApp messages
- phone calls
- informal referrals

This results in:

- inefficient service matching
- missed opportunities
- lack of automation
- poor user experience

At the same time, users struggle to find:

- **reliable services quickly**
- **availability in real time**
- **trusted providers nearby**

Problem Statement

Build an **Agentic AI System** that automates the **end-to-end lifecycle of a service request** — from user intent to booking and follow-up.

Your system must:

1. Understand user service requests (in natural language)
2. Identify relevant providers using location/context
3. Select or recommend the best provider
4. Simulate booking and confirmation
5. Handle follow-up interactions
6. Show complete reasoning and workflow execution

Mandatory Requirement: Google Antigravity

Teams **MUST use Google Antigravity** as the core platform to:

- orchestrate agent workflows
- manage multi-step reasoning
- integrate tools (Maps, Search, APIs)
- execute actions (booking, notifications, etc.)

Use of external LLMs is allowed, but **Antigravity must be central to system logic and orchestration**.

Example User Scenario

User input (Roman Urdu / Urdu / English):

“Mujhe kal subah G-13 mein AC technician chahiye”

Expected Output

Service Request:
AC Technician
Location:
G-13
Time:
Tomorrow morning
Recommended Provider:
Ali AC Services (2.1 km away)
Reasoning:
Closest available provider with high rating
Simulated Booking:
- Slot booked: 10:00 AM
- Confirmation sent
Follow-up:
Reminder scheduled 1 hour before appointment

System Requirements

1. Intent Understanding

- Process natural language input
- Support:
  - § Urdu
  - § Roman Urdu
  - § English
- Extract:
  - § service type
  - § location
  - § time

2. Provider Discovery

- Use:
  - § mock dataset OR
  - § Google Maps / Places APIs
- Identify:

§ nearby providers
§ service category match

3. Matching & Ranking

- Rank providers based on:

  § distance
  § availability
  § rating (simulated or real)
- Provide **clear reasoning for selection**

4. Decision & Recommendation

- Select best provider OR show top options
- Explain decision in simple terms

5. Action Simulation (CRITICAL REQUIREMENT)

System must simulate:

- booking confirmation
- provider assignment
- scheduling

Simulation can include:

- updating a mock booking system
- creating a confirmation message
- writing to a database/spreadsheet
- generating a booking receipt

6. Follow-Up Automation

- Simulate:

  § reminders
  § status updates
  § completion confirmation

7. Agentic Workflow (MANDATORY)

System must demonstrate:

- multiple agents OR structured reasoning pipeline
- planning → decision → action → follow-up
- traceable logs of:

  § decisions
  § tool usage
  § action execution

Deliverables

1. Working Prototype with Mobile App (MUST) and Web App (Optional)

2. Demo Video (3–5 minutes)

Must clearly show:

- user input
- system understanding
- provider matching
- booking simulation
- follow-up workflow

3. Agent Trace / Logs

- reasoning steps
- agent interactions
- action execution logs

4. Documentation (README)

Include:

- system architecture
- how Antigravity is used
- APIs/tools used
- assumptions and limitations

Evaluation Criteria

1. Use of Google Antigravity — 25%

- Core orchestration handled via Antigravity
- Effective use of tools (Maps, APIs)
- Demonstrates planning + execution

2. Agentic Reasoning & Workflow — 20%

- Multi-step reasoning
- Logical flow from request → decision → action
- Evidence of autonomy

3. Matching Quality & Decision Logic — 20%

- Relevant provider selection
- Clear ranking criteria
- Strong reasoning behind decisions

4. Action Simulation & Execution — 15%

- Booking process realistically simulated
- Clear system state change (confirmation, scheduling)
- End-to-end workflow demonstrated

5. Technical Implementation — 10%

- Clean architecture
- API/tool integration
- Robust handling of edge cases

6. Innovation & UX — 10%

- Creative approach
- Intuitive interface
- Clear and engaging demo

Important Guidelines

- This is NOT a simple listing or booking app
- Focus on **agentic automation, not UI complexity**
- At least one booking must be **simulated end-to-end**
- Must demonstrate **reasoning + decision-making**
- Use **mock data if real APIs are unavailable**
- Avoid use of real personal/sensitive data

Challenge 3: Crisis Intelligence & Response Orchestrator (CIRO)

Challenge Overview

Metropolitans, globally and in Pakistan, frequently faces **localized crises** such as:

- urban flooding
- heatwaves
- road blockages
- accidents
- infrastructure failures

However, response systems are:

- fragmented
- reactive
- slow to coordinate

Critical signals (social media, maps, weather, reports) exist — but are **not converted into actionable decisions in real time**.

Problem Statement

Build an **Agentic AI System** that:

1. Ingests **multi-source signals**
2. Detects emerging crisis situations
3. Generates **coordinated response actions**
4. **Simulates execution of those actions**
5. Shows impact of decisions

Mandatory Requirement: Google Antigravity

Teams MUST use **Google Antigravity** to:

- orchestrate multi-agent workflows
- plan and execute decisions
- integrate tools (Maps, Search, APIs)
- simulate coordinated actions

Example Scenario

Input Signals:

- Social media:

“Flash flood happening at George Town for past 30 mins" **OR**

“G-10 mein pani bhar gaya hai, gaariyan phans gayi hain”

- Weather:
    - § heavy rainfall alert
- Maps:
    - § traffic congestion spike

Expected Output

Detected Situation:
Urban flooding (G-10/ George Town)

Confidence:
High

Impact:
- Traffic blocked
- Vehicles stranded

Recommended Actions:
- Redirect traffic via alternate routes
- Dispatch emergency services

Simulated Execution:
- Route updated on map
- Alert sent to users
- Emergency ticket created

Outcome:
Reduced congestion in simulation

System Requirements

1. Multi-Source Input Processing
- Accept:
    - § text inputs (complaints, posts)
    - § simulated APIs (weather, traffic)
- Handle noisy, informal language

2. Event Detection
- Identify:
    - § anomalies
    - § clusters
    - § crisis signals

3. Reasoning & Situation Analysis
- Combine signals to:
    - § infer situation
    - § estimate severity
- Provide **confidence level + explanation**

4. Action Planning

- Generate coordinated response actions:
  - § routing
  - § alerts
  - § resource allocation

5. Action Simulation (CRITICAL)

System must simulate:

- traffic rerouting
- emergency dispatch
- alerts/notifications

Simulation examples:

- updating mock map routes
- generating emergency tickets
- sending simulated alerts
- updating system status

6. Outcome Visualization

Show:

- before vs after scenario
- impact of actions
- system logs

7. Agentic Workflow (MANDATORY)

System must demonstrate:

- multiple agents OR structured reasoning pipeline
- interaction between agents
- planning → decision → execution

Deliverables

1. Working Prototype with Mobile App (MUST) and Web App (Optional)

2. Demo Video (3–5 minutes)

Must show:

- multi-source input
- detected crisis
- action planning
- simulated response
- outcome

3. Agent Trace / Logs

- reasoning steps
- agent decisions
- action execution

4. Documentation (README)

Include:

- system architecture
- Antigravity usage
- tools/APIs used
- assumptions

Evaluation Criteria

1. **Use of Google Antigravity — 25%**
    - Core orchestration handled via Antigravity
    - Multi-agent planning + execution
    - Tool integration
2. **Agentic Reasoning & Coordination — 20%**
    - Multi-agent interaction
    - Logical reasoning
    - Decision-making quality
3. **Situation Detection & Analysis — 20%**
    - Accuracy of event detection
    - Quality of insights
    - Clear explanations
4. **Action Planning & Simulation — 15%**
    - Realistic response actions
    - Effective simulation
    - Clear system state change
5. **Technical Implementation — 10%**
    - Clean architecture
    - API integration
    - robustness
6. **Innovation & UX — 10%**
    - creativity
    - usability
    - demo clarity

Important Guidelines

- Use **simulated datasets/APIs where needed**
- Avoid real sensitive data
- Focus on **decision-making, not just visualization**
- Must show **end-to-end workflow**

Challenge 4: The Mobile App Alchemy: Agentic Game Quest

(Genre-Agnostic: Hyper-Casual / RPG / Puzzle / Strategy/ Single/ Multiplayer)

Challenge Overview

Mobile gaming is evolving beyond static loops and predictable mechanics. The most compelling games today aren't just played; they are experienced through systems that learn, adapt, and surprise. This challenge invites you to build a next-generation mobile game that leverages **Agentic AI** to create a deeply engaging, "just-one-more-round" experience.

Whether your game is a 60-second puzzle, a deep-state strategy sim, or an immersive RPG, the goal is to build a core loop so seamless and rewarding that players find it impossible to put down.

Problem Statement

Build a mobile game across **any genre** powered by **Google Antigravity** where:

- **Dynamic Flow:** The game adapts its difficulty, narrative, or environment in real-time based on player behavior.
- **Intelligent Mechanics:** Use AI to move beyond basic scripts, creating "living" game systems that respond uniquely to every session.
- **High Retention Loop:** Design a gameplay cycle—**Action** → **Feedback** → **Reward**—that is polished, responsive, and keeps the player coming back.
- **Optional Innovations:** While not mandatory, you are encouraged to experiment with multimodal inputs (camera/audio) or social/multiplayer features if they enhance the core loop.

Core Requirement: Google Antigravity

Teams **MUST** use Google Antigravity as the primary orchestrator to:

- **Drive Gameplay Logic:** Manage the "brain" of the game, including NPC behavior, level generation, or rule-setting.
- **Agentic Reasoning:** Use structured agent workflows to handle complex game states (e.g., an AI "Dungeon Master" or a "Smart Rival").
- **State Management:** Ensure game progression, scoring, and rewards are handled securely and intelligently within the Antigravity environment.

The "Agentic" Edge

What makes your game different? Use Agentic AI to solve common gaming hurdles:

- **Infinite Variety:** Instead of 10 handcrafted levels, use agents to generate endless, balanced challenges.
- **Personalized Rivalry:** Create AI opponents that learn from the player's specific tactics and force them to evolve.
- **Living Narratives:** In RPGs or adventures, let agents drive dialogue and world events that aren't hard-coded.

System Requirements

1. The "Core Loop" Engine

- The game must demonstrate a clear and rewarding primary action (e.g., matching tiles, clearing a room, managing resources).
- The transition between rounds or levels must be fluid and encouraging.

2. Agentic Workflow (MANDATORY)

- The system must show structured reasoning. You should be able to demonstrate how the AI "decided" to change a difficulty setting, generate a specific item, or move an NPC.

3. Visual Feedback & State

- Real-time updates to scores, leaderboards, or player attributes.
- High-quality UI/UX that feels responsive to user input.

4. Validation & Fair Play

- The AI agent must act as a "referee," ensuring that logic is followed and rewards are earned fairly.

Deliverables

1. **Working Prototype:** A Mobile Gaming App.
2. **Demo Video (3 mins):** Focus on the gameplay experience and how the AI makes it uniquely engaging.
3. **Architecture Map:** A breakdown of your Antigravity workflows and agent roles.
4. **README:** Explanation of the game's "hook" what makes the gameplay loop rewarding?

Evaluation Criteria

Criteria	Weight	Description
Antigravity Execution	**30%**	Deep integration of Antigravity for core logic and agent workflows.
Gameplay Engagement	**25%**	How compelling is the game? Is the core loop rewarding and polished?

Agentic Innovation	**20%**	Does the AI feel "smart"? Does it add value beyond traditional coding?
Technical Polish	**15%**	Stability, UI/UX fluidity, and overall performance.
Concept & Originality	**10%**	Creativity in using AI to reinvent a genre or create a new one.

Important Guidelines

- **Focus on the Hook:** Prioritize a smooth, rewarding user experience over complex secondary features.
- **Agentic Transparency:** Be prepared to show your agent traces, we want to see the "thinking" behind the fun.